Low Code Development Cycle Investigation



Małgorzata Pańkowska

Abstract Technology plays an important role in the advancement of end-user development of software applications. It supports the way requirements are elicited, collected, analyzed, and processed into functionalities and non-functionalities in information systems. Technology enables end-users to create their own application for particular usage. This paper aims to present low code application development methodology resulting from practical experience as well as from the literature survey. The paper findings present that the low code development cycle (LCDC) is different in comparison with traditional or agile software development methods. The LCDC should emphasize the role, competencies, and experience of the end-user, who is a low code developer. This paper consists of two main parts. The first part covers literature surveys on contemporary approaches to citizen software development and low coding. The second part covers the LCDC ecosystem and process, which reveals the main active role of end-user and supportive roles of other project stakeholders, i.e., analysts, software engineers, testers, integrators, and other facilitators.

Keywords Citizen development · Low code · Ecosystem · Process · Life cycle

1 Introduction

Collaboration in the end-user software development community involves a variety of participatory methods and stakeholder objectives that do not necessarily align with those in conventional information system life cycle development collaboration. Particularly, the end-user (i.e., citizen) software development requires creativity of users, their engagement, flexibility, and decision-making throughout a project. Beyond that, an infrastructure is needed that supports communication, tooling, and software application implementation. The approach of involving in the development

M. Pańkowska (✉)
Department of Informatics, University of Economics in Katowice, 1 Maja, Silesia, 40-287, Katowice, Poland
e-mail: pank@ue.katowice.pl

© The Author(s) 2024

X.-S. Yang et al. (eds.), *Proceedings of Ninth International Congress on Information and Communication Technology*, Lecture Notes in Networks and Systems 1011,
https://doi.org/10.1007/978-981-97-4581-4_19

of end-users is increasingly used and is known as citizen development. It requires usage of specialized Computer Aided Software Engineering (CASE) tools, i.e., Low Code Development Platform (LCDP). Citizen software development fosters an open and participatory approach to application implementation, reducing the distance between end-user and software engineer, and contributing to the goals of a business unit. End-user is allowed to construct forms of logic for solving problems and to convert the constructed form of logic into built software. The low-code applications are implemented and exploited for various purposes in different disciplines, however, elaboration of a common methodology for low-code software life cycle development is still a challenge. First, the heterogeneity of citizens, i.e., end-users is particularly evident in various business organizations. They are in big as well as in small business units. They have various competencies, i.e., knowledge, skills, social competencies, experiences, habits, preferences, risk averse, and willingness to learn and to be innovative. Hence, a generalization of their requirements is almost impossible, and by definition low code applications are preferably developed by individuals and for their individual usage. That situation reveals problems of low code software maintenance. However, some end-users can be interested in dissemination of their software development activities for an enrichment of the citizen development as a method. Hence, they provide software documentation and work on integration of their application with other parts of the business information systems, i.e., legacy systems, in a business organization. Taking into account the basic duties of end-users and therefore their limited time for software development, the citizen development calls for inclusivity, which is achieved through selected practices, tools, and methodological procedures. Particularly, the computerized tools that serve the practical implementation of citizen software are to be supported by guidelines and highly intuitive. End-users usually have no time to learn, but they want to realize their requirements as quickly as possible.

Communication in the field of citizen software development means more than just publishing results. Short time success motivates further work, but on the other side—failure results in disappointment and discourages further involvement. Hence, the citizen developers need LCDP facilitators, who help in learning and the application optimization. There are also various attitudes of top managers, who may support, but, on the other side, can discourage business employees to individual application development. Successfully, an increasing number of institutions, including government agencies, and big companies, are showing an interest in the field of citizen software development. Surprisingly, they believe that the low-code software can be treated as a prototype of further professionally developed applications. Through prototyping, end-users are able to reveal their real expectations. Hence, the process of requirement elicitation for professional IT companies is much easier than in a situation, when the end-user does not want to be involved in an interviewing or software customization, and deployment process. Beyond that, small and medium enterprises (SMEs) are interested in citizen software development because of the lower cost of software prepared by end-users, i.e., software prosumers, who are doing the applications for themselves and by themselves. Frequently, SMEs financial conditions exclude

investment in professional software available in cloud or on-premise, but creativity of end-users and access to open source tools permit to fulfill the requirements.

This study aims to support a low code software development methodology. Author focuses on identification of low code application ecosystems as well as on modeling processes covering low-code software development life cycle. In this contribution, the author emphasizes the particular role of the end-user. The rest of that paper includes the following sections. First, a literature survey has been done to present actual justification of low-code software development. Next, the low-code software development ecosystem in Archimate notation is presented, and finally, the Business Process Model and Notation (BPMN) process of Low Code Development Life Cycle (LCDLC) is included.

2 Theoretical Background of Low Code Development

The Internet democratization refers to the rapid growth of access to Information Communication Technologies (ICTs). The Internet users have access to information as well as to software development tools. According to Haklay et al. [8], citizen science means public participation in a scientific research activity. By analogy, citizen software development is defined as a software project conducted by amateurs or nonprofessional designers. Citizen software development creates opportunities for conducting investigations on requirements elicitation, co-designing, and co-testing of low-cost sensors, as well as on collecting and co-analysing data, and co-developing data applications. Volunteerism is important to the understanding of citizen software development, which presents a step forward in the democratization of application implementation. That approach brings new knowledge to particular domains as well as to ICT. Citizen science has been for years developed in the do-it-yourself (DIY) culture for pragmatic reasons. For example, over the past 30 years, Visual Basic applications have been implemented by laypersons in business for their own usage. That approach to software development is also known as information system (IS) prosumption, i.e., production for own exploitation.

According to Senabre Hidalgo et al. [26], public participation in science includes participatory action research (PAR) and the involvement of various social organizations, i.e., chamber of commerce, consumer associations, non-profit organizations, social groups, and environmentalists' associations. Some of them are low-budget business units, looking for relatively low-cost investments in the ICTs. Although traditionally, an IT professional is a software project manager and end-users work as data gatherers and requirements' providers, in the citizen software development approach end-users are able to reveal their real problems, because they take active role and software engineers are co-designers and facilitators in that process. Facilitators are expected to create the necessary conditions for equitable and free speaking, and to support collective decision-making during project meetings. Senabre Hidalgo et al. [26] argue that a facilitator is in charge of suggesting the materials and ensuring discussions among project partners. End-users' participation in software projects has

been defined as the degree to which the user is involved in producing and exploitation of the project products. User involvement refers to the subjective psychological state reflecting the importance of a given system.

Participatory Design (PD) has focused on the designing of user applications or the co-realization of information systems. In this approach, users are the best to determine how to automate their work, and the designers are just consultants. PD is realized through workshops where users can feel free to express their ideas concerning the designed applications [28]. User-Centered Design (UCD) is a philosophy that is based on studying the end-users' needs and interests. Not only the profiles, activities and environments of users are being investigated, but also their goals and values [20]. UCD is a process focusing on usability throughout the entire development process and further throughout the system life cycle. This philosophy highlights the role of prototyping, which must be used to visualize and evaluate ideas and design solutions in cooperation with the end users in a real context [7]. The method of User Experience (UX) Design is defined as the creation of the subjective relationship between user and application or device. That design is about considering the user, the task, and the context, i.e., environment of work within the company culture. It aims to enhance the entire experience and emotions resulting from the use of a product, system, or service [1]. Actor Network Theory (ANT) was developed by Callon and Latour to describe the creation and evolution of socio-technical systems and to focus on the dynamics of relationships among main actors (i.e., end users) and networks (i.e., business managers and ICT people) [3]. Those theories, although useful and supportive for software project management, have not emphasized the main role of end-users in the application implementation process.

Lately developed agile methods and DevOps approach are also combined with the LCDP usage. Lebens and Finnegan [12] have proposed to use the LCDPs to teach Agile Methodology. This approach was rational, because the agile methods focus on the end user involvement in fast result generating, prototyping, and software validation. University students can use the Microsoft Power Apps platform to develop their competences in agile software development. Rather old, but still valid approach can be the Rapid Application Development (RAD), which is a software development methodology promoting the rapid release of a software prototype. This agile approach utilizes the user feedback from the prototype for further improvement of the product. Therefore, the low-code prototypes can be included in projects, where developers use the RAD tools, i.e., LCDPs, Joint Requirement Planning (JRP), and the Joint Application Development (JAD). JAD uses organized and intensive workshops to bring together project stakeholders, i.e., sponsors, users, analysts, and programmers to jointly define and design information systems. Low code prototyping aims to make end-user ideas tangible, realizable by creating a first implementation. The low code development platform can also be used in the Design Thinking (DT) methodology. Although it is still a challenge, the DT is a methodology that uses co-designing and various problem solving techniques to ensure an alignment of business needs with what is technologically feasible. The DT methodology incorporates an empathy for the user, rapid prototyping and abductive reasoning [4].

Taking into account that citizen software development is a way to democratize the application implementation, this study has formulated two research questions, i.e., What are the Low Code Development (LCD) Methodologies? What are the Citizen Development (CD) methodologies? The author has proposed the research structure consisting of two parts, i.e., a literature survey, and the LCDLC proposal. Answers that research questions have been received through the literature surveys. Therefore, the author reviewed the following repositories: Scopus, Association for Information Systems Electronic Library (AISeLib), Springer Link, IEEEXplore, and Sage Journals. The search phrase was as follows: "citizen development" AND "low code development". In general, the author has found 1272 publications released in 2010–2023. The publication sources were not restricted. Articles were identified and screened for relevance through abstract, title, and keywords. Next, 700 publications were excluded because of lack of access to the full text and too less valuable information about the research findings in the abstracts. Hence, 572 publications were reviewed. However, that number of publications was further reduced, because mostly, they were not strictly about the citizen software development methodology, nor about the low code development methodology. In the next step, 52 publications were studied. That publications cover mostly discussions on the low code platforms, their applicability strengths and weaknesses, as well as on the LCDP's experimental usage results.

As the LCDPs provide a graphical user interface to drag and drop with little effort and in a user-friendly way, they are easy to learn and speed up the application development. Martinez-Lasaca et al. [15] argue that the LCDPs enable prototyping, hence project stakeholders have an opportunity to find a compromise solution in a conflicting situation. The LCDPs are particularly preferred in domains that have quite well identified procedures and processes, and in which stakeholders are interested in the business process automation. The LCDPs enable data handling and workflow automation based on business processes modeled in the Business Process Model and Notation (BPMN) as a reference modeling language [24, 29]. The survey results show that the domains where low code development is mostly used are agriculture [6, 18, 19], Internet of Things (IoT) application exploitation [5], manufacturing industry [23, 25], pharmaceutical enterprises [17], high school education [14], and Enterprise Resource Planning (ERP) cloud services and components [27]. Brouzos et al. [2] discuss the low code development of cyber-physical systems such as robots and sensors. Martins et al. [16] present a model designed to transform ChatGPT into a low code developer. Leading vendors in the LCDP field are Appian, Mendix, Microsoft, OutSystems, Salesforce, and Service Now [9].

3 Low Code Development Life Cycle Model

The LCD literature survey allows for an identification of a gap concerning the low code development life cycle modeling from the end user perspective. Rokis and Kirikova [21] have identified the LCD principles, i.e., selection of the right platform, visual application outline, usage of predefined components, empowerment of citizen developers, establishing a project team and governance, and iterative development life cycle. They have noticed some necessities, i.e., feasibility analysis, data modeling, defining of the user interfaces, implementation of business logic, integration of external services, testing and deployment. Some of those necessities result from the LCDP guidelines, others are included in the traditional system development life cycle approach. Kuruoglu et al. [10] have presented the low code development methodology based on process modeling and prototyping. Ruscio and Pierantonio [22] have proposed a low code development life cycle consisting of some stages, i.e., data modeling, user interface design, business logic implementation, integration of external services, testing and deployment, and customer feedback. Krejci et al. [11] have recognized that software development on a LCDP consists of similar process phases as in traditional projects. They have identified the following LCD project phases: preparation, generation, improvement and implementation, evaluation, deployment, and improvement in next iterations. As presented in the literature, low-code software development stages are discussed on a high level of preliminary investigation. Unfortunately, the authors do not emphasize the unique role of end-users. However, this study focuses on the LCD process, in which the end-user plays the main roles (see Fig. 2). That LCD process is included in the LCD ecosystem modeled in the Archimate language (see Fig. 1).

The complexity and importance of the LCD software projects, which include using hardware equipment and the LCDP software resource to create products and services encourage the development of the whole LCD ecosystem (see Fig. 1). The concept of ecosystem is quite comfortable, because the developers need not to concentrate on a particular enterprise, but they can focus on identification of the project goals, drivers, stakeholders, principles, business actors, processes, material resources, software and hardware components. The ecosystem planning is always located in a certain strategic motivation context, which is necessary to justify the business requirements and the need to design and implement the software application. That context includes business justification of the LCDP usage. In Fig. 1, the context is limited to goals, drivers, and principles. The non-professional may collaborate with professionals in all stages, but mostly they alone contribute to software implementation. Their goals are as follows (based on [13]):

1. Self-direction, competencies enrichment—"I want to learn".
2. Power through exercising control over people, data, and material resources—"I want to gain recognition and status".
3. Commitment to realization of the business unit strategy—"I want to improve my organization works".
4. Efficiency, reduction of the ICT costs—"I can implement cheaper application".

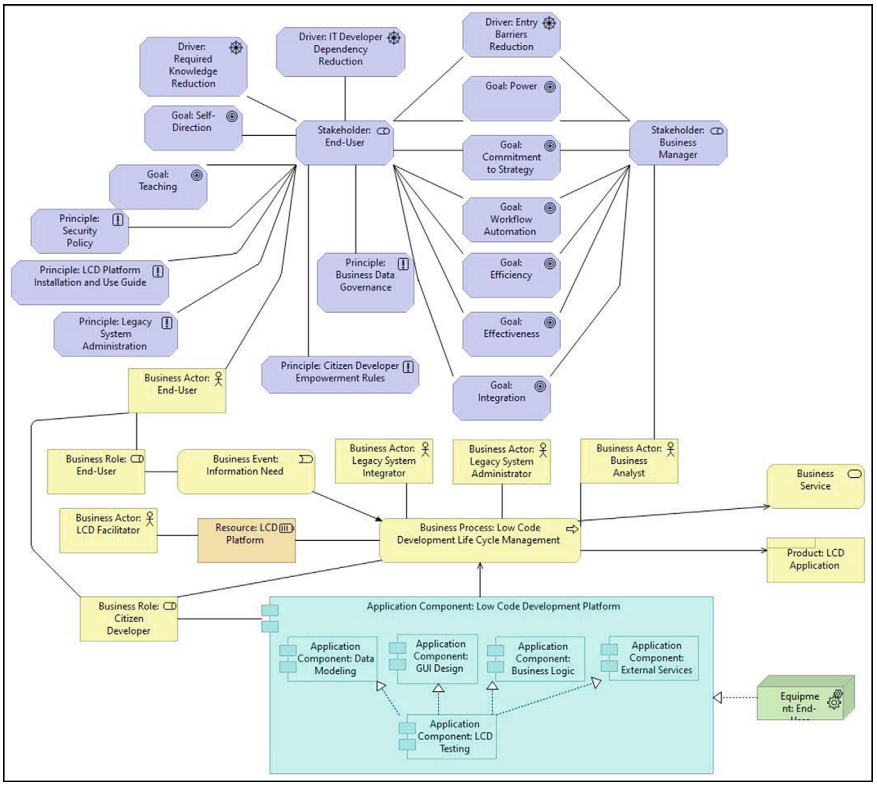


Fig. 1 The low code development ecosystem model in archimate notation

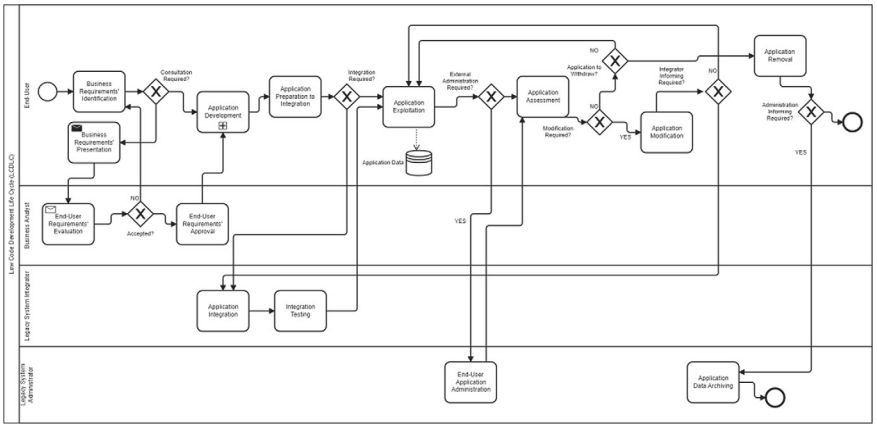


Fig. 2 The low code development life cycle process in notation BPMN

5. Workflow automation—"I want to do it automatically".
6. Effectiveness, elicitation of unique requirements—"I can implement an unique functionality, which is not available in a Commercial of The Shelf (COTS) software".
7. Teaching and providing an educational opportunity to others—"I want to share my knowledge and experience".
8. Integration—"I want to have my application integrated with others".

The identified goals and drivers are linked to the stakeholders, i.e., End-User and partially to the Business Manager. The drivers stimulating low-code development are the following: reduction of entry barriers for software development, reduction of required knowledge for application development, and reduction of dependence on IT developers [9]. Some principles are assumed to be important to the End-user. They are as follows: Security Policy, LCDP Installation and Use Guide, Legacy System Administration, Business Data Governance, and Citizen Developer Empowerment Rules. The LCD ecosystem model takes into account two general scenarios. The first scenario assumes that a citizen developer is working for a separate business unit, e.g., SME, micro company. Then the developer is the only person responsible for data governance and application implementation, maintenance, and administration. The second option covers situations, when a citizen developer is working for a SME or big company. Then the application can be combined with other business software, in this ecosystem model named legacy systems. In the second case, stakeholders have to solve problems of the application integration and administration, as well as data governance. Generally, the LCD ecosystem model covers just one process named Low Code Development Life Cycle Management. The Archimate models allow for high- level strategic modeling of technology implementation issues. Details are included in Figs. 2 and 3. The detailed design should be elaborated in modeling languages, e.g., SysML, UML. Implementation can be done in a LCDP, e.g., OutSystems, Webcon (webcon.com). The LCD ecosystem model assumes that a business event triggers a process, which results in providing business service and product, i.e., the low code application. The process realization depends on the selected LCDP, which includes components for data modeling, user interface design, business logic modeling and implementation, and external services integration. In both scenarios, the LCDP is assumed to be located on the End-User Computer. In this way, the author emphasizes the specific role of citizen developer and meaning of the LCDP, which is not widely used by professional software engineers. That model includes another actor, who is the LCDP Facilitator responsible for helping the End-User in learning, designing, and maintenance of the low code application. The End-User as an actor has two roles: End-User and Citizen Developer. The LCDLC process is included in Figs. 2 and 3.

The LCDLC process is initiated by the End-User, who is inspired by the personal information needed to develop an LCD application. That requirement can be consulted with the Business Analyst, but in another option, it can be developed independently. The same approach is applied to all other tasks in the LCD cycle. The important issue, which arrives in that process, is the governance and retention

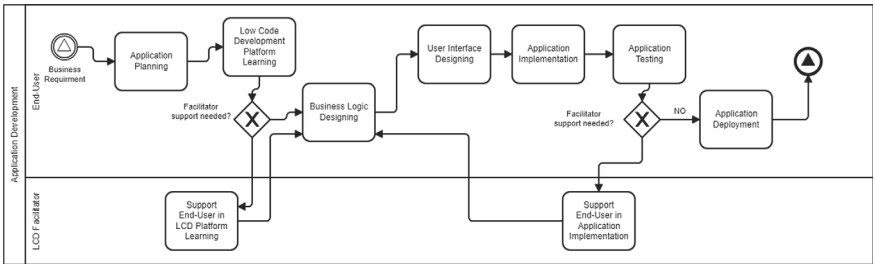


Fig. 3 The application development sub-process in the LCDLC process in notation BPMN

of the application data. The business data should be eventually secured and archived by the business representative, e.g., Administrator. Hence, the company should have elaborated the Citizen Developer Empowerment Rules (see Fig. 1), which explains the data governance principles. For SMEs and micro companies, when the Citizen Developer is the business owner and administrator, they establish the data governance rules. However, the situation when the application and data are just for one End-User is also possible. The LCDLC process (see Fig. 2) includes one sub-process named Application Development (see Fig. 3).

Through this LCDLC process model, the author wants to emphasize the main role of the End-User in the application development process. This opportunity was created just because of the LCDPs availability. This process encourages further questions, e.g., the data governance, involvement of the business analyst, the professional software engineers, system administrators, system integrators, and testers in the application development, as well as the LCDP maintenance costs and procedures. Beyond the lack of governance rules, there are some other inhibitors, i.e., unsolved issues of scalability, limited portability significantly inhibit the LCDP adoption, as well as high prices of commercial tools, lack of customization, vendor lock-on, and lack of knowledge about LCDPs [9].

4 Conclusions

Although end-users have opportunities to use various low code development platforms, the different technical capabilities and multiple scenarios of usage still create new challenges. The literature survey revealed that IT professionals use the LCDPs under control of IT companies. In this paper, the author presents another process of the LCDP usage, which is less formalized, and its heterogeneity hinders its generalization.

References

1. Beccari MN, Oliveira TL (2011) A philosophical approach about user experience methodology. In: Markus A (ed) Design, user experience and usability, theory, methods, tools and practice. Springer, Heidelberg, pp 13–22
2. Brouzos R, Panayiotou K, Tsardoulias E, Symeonidis A (2023) A low-code approach for connected robots. J Intell Rob Syst Theory Appl 108(2):28
3. Callon M, Latour B (1981) Unscrewing the big leviathan: how actors macro-structure reality and how sociologists help them to do so. In: Knorr-Cetina KD, Cicourel AV (eds) Advances in social theory and methodology: towards an integration of micro and macro-sociologies. Routledge and Kegan Paul, London, pp 277–283
4. Chasanidou D, Gasparini AA, Lee E (2015) Design thinking methods and tools for innovation. In: Marcus A (ed) Design, user experience, and usability, design discourse. Springer, Cham, pp 12–23
5. Chaudhary HAA, Guevara I, Singh A, Schieweck A, John J, Margaria T, Pesch D (2023) Efficient model-driven prototyping for edge analytics. Electron (Switzerland) 12(19), Article number 3881
6. Fatouros G, Kousiouris G, Lohier T, Makridis G, Polyviou A, Soldatos J, Kyriazis D (2023) Enhancing smart agriculture scenarios with low-code, pattern-oriented functionalities for cloud/edge collaboration. In: Proceedings—19th International conference on distributed computing in smart systems and the internet of things, DCOSS-IoT 2023, pp 285-292, IEEE, Pafos
7. Gulliksen J, Goransson B, Boivie I, Persson J, Blomkvist S, Cajander A (2005) Key principles for user-centered systems design. In: Seffah A, Gulliksen J, Desmarais MC (eds) Human-centered software engineering—integrating usability in the software development lifecycle. Springer, Berlin, pp 17–37
8. Haklay MM, Dorler D, Heigl F, Manzoni M, Hecker S, Vohland K (2021) What is citizen science? The challenges of definition. In: Vohland K, land-Zandstra A, Ceccaroni L, Lemmens R, Perello J, Ponti M, Samson R, Wagenknecht K (eds) The science of citizen science, pp 13–34. Springer, Cham
9. Kass S, Strahringer S, Westner M (2023) Practitioners' perceptions on the adoption of low code development platforms. IEEE Access 11:29009–29034
10. Kuruoglu Dolu B, Cetinkaya A, Kaya MC, Nazlioglu S, Dogru AH (2022) MSDeveloper: a variability-guided methodology for microservice-based development. Appl Sci (Switzerland) 12(22):11439
11. Krejci D, Iho S, Missonier S (2021) Innovating with employees: an exploratory study of idea development on low-code development platforms. In: ECIS 2021 Research papers. 118. https://aisel.aisnet.org/ecis2021_rp/118. Accessed 21 Nov 2023
12. Lebens M, Finnegan R (2021) Using a low code development environment to teach the agile methodology. In: Gregory P, Lassenius C, Wang X, Kruchten P (eds) XP 2021, vol 419. LNBIP. Springer, Cham, pp 191–199
13. Land-Zandstra A, Agnello G, Selman Gultekin Y (2021) Participants in citizen science. In: Vohland K, Land-Zandstra A, Ceccaroni L, Lemmens R, Perello J, Ponti M, Samson R, Wagenknecht K (eds) The science of citizen science. Springer, Cham, pp 243–260
14. Luo J, Hou M (2023) Innovation of the higher education grassroots statistical reports system based on low-code development. In: 7th International conference on management engineering, software engineering and service sciences, ICMSS, pp 36–40
15. Martinez-Lasaca F, Diez P, Guerra E, de Lara J (2023) Dandelion: a scalable, cloud-based graphical language workbench for industrial low-code development. J Comput Lang 76:101217
16. Martins J, Branco F, Mamede H (2023) Combining low-code development with ChatGPT to novel no-code approaches: a focus-group study. Intelli Syst Appl 20:200289
17. Miyake T, Masuda Y, Oguchi A, Ishida A (2023) Strategic risk management for low-code development platforms with enterprise architecture approach: case of global pharmaceutical enterprise. Smart Innovation Systems Technol. SIST 357:88–100

18. Novales A, Mancha R (2023) How hortilux used low-code to develop its IoT digital service. Commun Assoc Inf Syst 53:924–937, 38
19. Oteyo IN, Scull Pupo AL, Zaman J, Kimani S, De Meuter W, Gonzalez Boix E (2023) Easing construction of smart agriculture applications using low code development tools. In: Longfei S, Bodhi P (eds) Mobile and ubiquitous systems: computing, networking and services. MobiQuitous 2022, pp 21–43. Springer, Cham
20. Raatikainen P, Pekkola S (2023) User-centredness in large-scale information systems implementation. In: 12th Scandinavian conference on information systems. 3. https://aisel.aisnet.org/scis2021/3. Accessed 23 Dec 2023
21. Rokis K, Kirikova M (2023) Exploring low-code development: a comprehensive literature review. Complex Syst Inf Model Q (36):68–86, 200
22. Ruscio, D., Pierantonio, A.: Supporting the understanding and comparison of low-code development platforms. In: 2020 46th Euromicro conference on software engineering and advanced applications (SEAA), pp.171–178. IEEE, Portoroz (2020).
23. Qu D, Zhang Y, Hu X, Dai W (2023) Contract-based design for low-code development in industrial edge applications. In: 2023 IEEE 32nd International symposium on industrial electronics (ISIE), pp 1–6. IEEE, Helsinki
24. Sahay A, Di Ruscio D, Iovino L, Pierantonio A (2023) Analyzing business process management capabilities of low-code development platforms. Softw Pract Experience 53(4):1036–1060
25. Schenkenfelder B, Salomon C, Buchgeher G, Schossleitner R, Kerl C (2023) The potential of low-code development in the manufacturing industry. In: IEEE international conference on emerging technologies and factory automation, ETFA, Code 193521. IEEE, Sinaia
26. Senabre Hidalgo E, Perello J, Becker F, Bonhoure I, Legris M, Cigarini A (2021) Participation and co-creation in citizen science. In: Vohland K, Land-Zandstra A, Ceccaroni L, Lemmens R, Perello J, Ponti M, Samson R, Wagenknecht K (eds) The science of citizen science. Springer, Cham, pp 199–218
27. Tang L (2022) ERP low-code cloud development. In: Proceedings of the IEEE international conference on software engineering and service sciences, ICSESS, pp 319–323
28. Torpel B, Voss A, Hartswood M, Procter R (2009) Participatory design: issues and approaches in dynamic constallations of use, design and research. In: Voss A, Hartswood M, Procter R, Rouncefield M, Slack RS, Buscher M (eds) Configuring user-designer relations. Springer, London, pp 13–30
29. Wang J, Qi B, Zhang W, Sun H (2021) A low-code development framework for constructing industrial apps. In: Communications in computer and information science. 1330 CCIS, pp 237–250

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

